

1차 방문 안내서 (안심구역 · v4)

컬러로 인쇄하세요 (검은 입력 상자·빨간 번호·초록/빨강 칸을 색으로 구분합니다).

1차 방문 날 안심구역 컴퓨터에서 할 일을 이 문서 하나에 모았습니다. 예전 문서 넷(상세판·따라하기판·그림안내·맥 리허설)은 이 문서로 합쳤고, 원본은 docs/archive/ 에 보관합니다.

부	누가 보나	내용
1부 한 장 요약	명령창이 익숙한 사람	전체 순서와 명령을 한 장에
2부 따라 하기	처음 해 보는 사람	한 장에 한 단계 (단계 0 ~ 8). 화면 그림과 "성공하면/이상하면"
3부 자세한 설명	왜 이렇게 하는지 궁금할 때, 결과 받은 뒤	구역을 나누는 이유, 확인 기준, 반출 규칙, 밖에서 합치는 법
4부 부록: 맥 리허설	방문 전에 연습할 사람	이 맥에서 가짜 데이터로 똑같이 해 보기

읽는 법 — 무엇을 입력하나

- 아래처럼 "입력" 이 붙은 검은 상자 안의 글자만 명령창에 그대로 칩니다 (글자·띄어쓰기·기호까지). 위의 "하는 일" 은 그 명령이 무엇을 하는지 적은 설명이라 치지 않습니다.

하는 일 (보기) 이 명령이 하는 일 — 이 줄은 입력하지 않음

입력 `python run_all.py`

- 문장이나 표 속의 `notepad config.py` 처럼 검은 칸도 입력할 명령입니다.
- 회색 칸(`DATA_ROOT_MAP`, `AREA_BBOX = None`, `D:\작업폴더\hbi>` 등)은 화면이나 파일에서 찾아볼 글자라 입력하지 않습니다. (메모장에서 바꿀 줄은 그 장의 그림과 설명대로 고칩니다.)
- `D:\작업폴더` 자리에는 단계 0 ④ 에 적은 주소를 칩니다.

꼭 지킬 것 세 가지

- 구역(A·B·C·D)마다 `AREA_BBOX` 한 줄만 바꾸고 `python run_all.py` 한 줄로 돌립니다.
- `EDGE_BUFFER` 는 바꾸지 않습니다 (500 그대로).
- `output_A ~ output_D` 만 반출합니다. `work` 폴더는 반출하지 않습니다.

1부. 한 장 요약

흐름: ① 준비(한 번) → ② 구역 A·B·C·D 네 번 → ③ 반출. v4 코드는 서로 이어진 길 묶음 가운데 가장 큰 하나만 계산하므로, 떨어진 네 동네 묶음을 분석 범위(AREA_BB0X)를 바꿔 가며 따로 돌립니다.

순서	할 일	명령 / 바꿀 줄
① 준비 1	담당자에게 폴더 4개 주소 받기: 수치지형도, DEM 5m, 국토정보필지(없으면 생략), 작업 폴더	
① 준비 2	시작 메뉴 → OSGeo4W Shell 열기	
① 준비 3	번들 풀기	<code>cd /d D:\작업폴더</code> → <code>python hbi_code_bundle_v4.txt</code> → <code>cd hbi</code>
① 준비 4	설정 파일에 주소 3개 (<code>r"..."</code> 의 <code>r</code> 은 그대로, 저장은 UTF-8)	<code>notepad config.py</code> → <code>DATA_ROOT_MAP, DATA_ROOT_DEM, DATA_ROOT_PARCEL</code>
① 준비 4 확인	주소 3줄이 받아 적은 주소인지	<code>findstr /b "DATA_ROOT" config.py</code>
① 준비 5	자료 점검: shp 파일 총 N개 0 아님, [있음] sidewalk_cl·road_cl·building, 지목 한글	<code>python 01_inspect.py</code>
② 구역마다 1	# 없는 AREA_BB0X 줄을 아래 표의 그 구역 줄로 바꾸고 저장 (# 예) ... 줄은 건드리지 않음)	<code>notepad config.py</code>
② 구역마다 1 확인	화면에 정확히 한 줄, 그 구역 줄과 같음. 두 줄이거나 값이 다르면 메모장으로 돌아가 고침	<code>findstr /b "AREA_BB0X" config.py</code>
② 구역마다 2	분석 (01·02·03·04·05·06 순서로 나옴. 06 은 필지가 있을 때만)	<code>python run_all.py</code>
② 구역마다 3	메모: 02 최대 연결망 노드 비율 (90% 이상), 03 데이터 경계 500m 이내 ...(x%) (100% 아님)	
② 구역마다 4	결과 폴더 이름 바꾸기	<code>ren output output_A</code> (B·C·D)
③ 마무리	AREA_BB0X = None 으로 되돌리기 (확인: 한 줄 AREA_BB0X = None), output_A~D 반출 신청 (work 는 반출 안 함)	<code>notepad config.py</code> → <code>findstr /b "AREA_BB0X" config.py</code>

구역	동네	메모장에 넣을 줄
A□	종로구·중구	AREA_BB0X = [194500, 548300, 203400, 560200]
B□	관악구	AREA_BB0X = [190100, 536300, 200000, 545000]
C□	광진구	AREA_BB0X = [203900, 546100, 211100, 553700]
D□	강서구	AREA_BB0X = [178200, 546400, 190500, 557200]

멈춤 규칙: !! ... 에서 중단 이 나오면 2부 단계 8 의 표대로 합니다. 표대로 해도 안 되면 그 구역은 **ren output output_A 실패** 하고 다음 구역으로 갑니다. 표에 없는 오류이면 멈추고, 오류 마지막 줄을 적어 옵니다.

2부. 따라 하기 (처음 해 보는 사람용)

한 장에 한 단계씩입니다. 장마다 맨 위에 "단계 0" ~ "단계 8" 이름이 있습니다(단계 5 는 두 장: 5, 5-2). 장마다 "할 일 → 따라 하기 → 화면 예시 → 성공하면/이상하면" 순서로 읽습니다.

그림 속 빨간 번호 ①②③ 은 그 장의 글에서 "그림 ①" 처럼 가리키는 곳입니다. "입력" 검은 상자와 검은 칸의 글자만 명령창에 칩니다 (표지의 "읽는 법"). 단 하나, D:\작업폴더 자리에는 단계 0 ④ 에 적은 주소를 칩니다.

단계 0. 준비물과 물어볼 것

이 단계에서 할 일

가져갈 것을 챙기고, 담당자에게 폴더 네 곳의 주소를 받아 적습니다.

1. 챙길 것: 신분증, 이 안내서, 펜
2. 안심구역 담당자에게 아래 네 가지를 묻고, 오른쪽 칸에 주소(폴더 위치)를 그대로 적습니다.

물어볼 것	받아 적을 주소
① 수치지형도 폴더 (지도 파일이 있는 곳)	
② DEM 5m 폴더 (높이 자료가 있는 곳)	
③ 국토정보필지 폴더 (서울 필지. 못 받으면 "없음")	
④ 작업 폴더 (우리 파일 hbi_code_bundle_v4.txt 를 둘 곳)	

✓ 성공하면

네 칸이 모두 채워짐 (③은 "없음"이어도 됨)

✗ 이상하면

주소를 모르겠다고 하면: "파일 탐색기에서 그 폴더를 열어 주세요"라고 부탁하고, 단계 3의 방법으로 주소를 복사

이 안내서에 나오는 낱말:

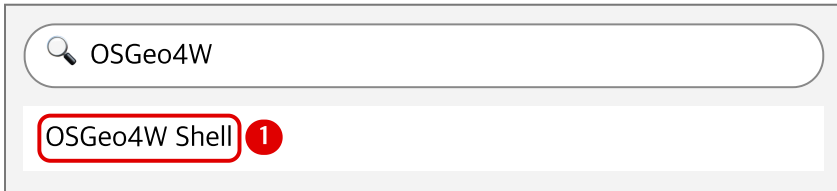
- **필지**: 땅 한 조각(지적도의 한 칸). **DEM**: 땅의 높이를 담은 자료. **shp**: 지도 파일의 한 종류
- **sidewalk_cl · road_cl · building** 같은 영어: 지도 파일 이름 (보도·도로·건물 등). 뜻을 몰라도 화면에서 글자만 맞춰 보면 됩니다
- **반출(반출 신청)**: 안심구역 안에서 만든 결과를 밖으로 가지고 나가겠다고 신청하는 것. 담당자가 확인한 뒤 내 보내 줍니다
- **번들**: 우리 분석 파일을 한 파일로 묶은 것 (hbi_code_bundle_v4.txt). **AREA_BBOX**: 분석 범위(계산할 지역)를 정하는 설정 줄
- **연결망 노드 비율**: 길이 끊기지 않고 한 덩어리로 이어진 정도. **데이터 경계**: 받은 지도 자료의 가장자리
- **PARCEL_ENCODING · CP949 · UTF-8**: 한글 글자를 읽는 방식 설정. 글자가 깨질 때만 바꿉니다. 그 밖의 영어(좌표계 prj 등)는 몰라도 됩니다
- **파일 탐색기**: 폴더 창. 키보드의 **윈도 키 + E** 를 함께 누르면 열립니다

단계 1. 명령창 열기

이 단계에서 할 일

명령창(글자로 컴퓨터에 지시하는 검은 창)을 엽니다.

1. 화면 왼쪽 아래 **시작** 단추를 클릭합니다.
2. 키보드로 **OSGeo4W** 를 칩니다 (검색 칸에 입력).
3. 목록에 나온 **OSGeo4W Shell** 을 클릭합니다(그림 ①). 검은 창이 뜹니다.
4. 이 안내서의 검은 **입력** 상자 속 명령은 검은 창에 **그대로 타자로 치고 Enter** 를 누릅니다. 다른 곳에서 글자를 복사해 왔다면, 검은 창 안에서 **마우스 오른쪽 클릭** 또는 **Ctrl+V** 로 붙여 넣을 수 있습니다.



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

명령이 끝나면 맨 아래에 **D:\작업폴더>** 처럼 **폴더 주소와 >** 가 다시 나타납니다 (단계 2 그림 ③ 에서 볼 수 있음). 이것이 "다음 명령을 쳐도 된다"는 신호입니다. 그 전에는 다음 명령을 치지 않습니다.

실행 중에는 검은 창 안을 클릭하지 않습니다. 클릭하면 "선택" 상태가 되어 실행이 멈춥니다. 창 맨 위 제목 앞에 **선택** 이 보이면 **Esc** 를 누릅니다. 마우스 휠로 위로 올려 보는 것은 괜찮습니다.

✓ 성공하면

검은 창이 뜨고, 맨 위 제목에 "OSGeo4W Shell" 이라고 쓰여 있음

✗ 이상하면

목록에 없으면 담당자에게 "QGIS 의 OSGeo4W Shell 이 어디 있나요?"라고 묻기. **실행 중에는 검은 창을 닫지 않습니다**

단계 2. 우리 파일 풀기

이 단계에서 할 일

작업 폴더로 가서 우리 파일(번들)을 풀고, 새로 생긴 hbi 폴더로 들어갑니다.

1. 아래 첫 번째 상자를 치고 Enter. `D:\작업폴더` 자리에는 단계 0 ④에 적은 주소를 칩니다.
2. 두 번째 상자를 치고 Enter. 완료: 파일 43개 가 나오면 됩니다(그림 ②).
3. 세 번째 상자를 치고 Enter. 맨 아래가 `...\hbi>` 로 바뀝니다(그림 ③).

하는 일 작업 폴더로 옮겨 갑니다 (`D:\작업폴더` 자리에 단계 0 ④ 주소)

입력 `cd /d D:\작업폴더`

하는 일 우리 파일(번들)을 풀어 `hbi` 폴더를 만듭니다

입력 `python hbi_code_bundle_v4.txt`

하는 일 새로 생긴 `hbi` 폴더 안으로 들어갑니다

입력 `cd hbi`

OSGeo4W Shell

```
D:\작업폴더>python hbi_code_bundle_v4.txt
완료: 파일 43개 → D:\작업폴더 ②
다음: cd hbi → python 01_inspect.py (README.md 참고)

D:\작업폴더>cd hbi

D:\작업폴더\hbi> ③
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

② 완료: 파일 43개, ③ 맨 아래가 `D:\작업폴더\hbi>`

✗ 이상하면

'python'은(는) 내부 또는 외부 명령... 이 나오면 단계 8 첫째 줄을 봅니다. 지정된 경로를 찾을 수 없습니다 가 나오면 단계 0 ④ 주소를 다시 확인하고 이 단계의 첫 번째 상자부터 다시 칩니다

단계 3. 설정 파일에 폴더 주소 넣기

이 단계에서 할 일

메모장으로 설정 파일(`config.py`)을 열어, 단계 0에 적은 폴더 주소 ①②③ 세 개를 넣습니다 (④는 단계 2에서 이미 씀).

1. 아래 상자를 치고 Enter. 메모장이 열립니다.
2. **Ctrl+F** → `DATA_ROOT_MAP` 을 쳐서 그 줄을 찾습니다.
3. 따옴표 " 안의 글자만 지우고 단계 0 ① 주소를 넣습니다(그림 ①). **따옴표 앞의 r**과 **따옴표 두 개는 지우지 않습니다**. 같은 방법으로 `DATA_ROOT_DEM` 에 단계 0 ② 주소를 넣습니다(그림 ②). `DATA_ROOT_PARCEL` 은 처음에 `None` 이라 따옴표가 없습니다: `None` 을 지우고 그림 ③ 처럼 `r"단계 0 ③ 주소"` 를 씁니다. ③ 이 "없음"이면 `None` 그대로 둡니다. 다른 줄(`DATA_ROOT_DEM1M`, `EDGE_BUFFER` 등)과 # 뒤 설명 글은 건드리지 않습니다.
4. 메모장 **오른쪽 아래가 UTF-8** 인지 보고 **Ctrl+S** 로 저장합니다 (다르면 단계 8 표). 메모장은 닫아도 됩니다. 저장한 뒤 단계 3-2 로.

하는 일 설정 파일 `config.py` 를 메모장으로 엽니다

입력 notepad `config.py`

바꾸기 전

```
config.py - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

DATA_ROOT_MAP = r"D:\data\수치지형도"      # ★ 수치지형도 .shp 파일들이 있는 폴더
DATA_ROOT_DEM = r"D:\data\DEM5m"          # ★ DEM 5m .img 파일들이 있는 폴더
...
DATA_ROOT_PARCEL = None                    # 국토정보필지 폴더 (2회차에 승인되면 경로 입력)
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

바꾼 후 (주소는 예시)

```
config.py - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

DATA_ROOT_MAP = r"E:\제공자료\수치지형도" ① # ★ 수치지형도 .shp 파일들이 있는 폴더
DATA_ROOT_DEM = r"E:\제공자료\DEM5m" ②     # ★ DEM 5m .img 파일들이 있는 폴더
...
DATA_ROOT_PARCEL = r"E:\제공자료\필지" ③    # 국토정보필지 폴더 (2회차에 승인되면 경로 입력)
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

주소 복사하기: 파일 탐색기에서 그 폴더를 열고 주소창 빈 곳을 클릭 → 주소가 파랗게 선택됨 → **Ctrl+C** → 메모장 따옴표 안에서 **Ctrl+V** (그림 ④).

```
파일 탐색기
E:\제공자료\수치지형도 ④
...
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

세 줄의 따옴표 안이 받아 적은 주소로 바뀌고 저장됨

✗ 이상하면

`r` 이나 따옴표를 지웠으면 그림처럼 `r"..."` 로 되돌림. 저장을 잊으면 단계 4에서 `shp` 파일 총 0개 가 나옴

단계 3-2. 고친 줄 확인하기

이 단계에서 할 일

설정 파일에서 주소 줄만 골라 보고, 받아 적은 주소가 들어갔는지 확인합니다.

1. 아래 입력 상자를 치고 Enter. `DATA_ROOT` 로 시작하는 줄 네 개만 나옵니다 (그림).
2. 그림 ① `DATA_ROOT_MAP`, 그림 ② `DATA_ROOT_DEM`, 그림 ③ `DATA_ROOT_PARCEL` 의 따옴표 안이 단계 0 에 적은 주소인지 한 글자씩 봅니다. `DATA_ROOT_DEM1M` 줄은 `None` 그대로면 됩니다.
3. 다르면 `notepad config.py` 로 돌아가 고치고 저장한 뒤, 이 단계를 다시 합니다.

하는 일 `config.py` 에서 `DATA_ROOT` 로 시작하는 줄만 골라 보여 줍니다 (확인용, 아무것도 바꾸지 않음)

입력 `findstr /b "DATA_ROOT" config.py`

주소는 예시

```
OSGeo4W Shell
D:\작업폴더\hbi>findstr /b "DATA_ROOT" config.py
DATA_ROOT_MAP = r"E:\제공자료\수치지형도" ① # ★ 수치지형도 .shp 파일들이 있는 폴더
DATA_ROOT_DEM = r"E:\제공자료\DEM5m" ② # ★ DEM 5m .img 파일들이 있는 폴더
DATA_ROOT_DEM1M = None # ★ DEM 1m 폴더 (비교 분석용, 안 쓰면 None 그대로)
DATA_ROOT_PARCEL = r"E:\제공자료\필지" ③ # 국토정보필지 폴더 (2회차에 승인되면 경로 입력)
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

네 줄이 나오고, ①②③ 의 따옴표 안이 받아 적은 주소 (필지가 "없음"이면 ③ 은 `None`)

✗ 이상하면

아무것도 안 나오면 `hbi` 폴더 밖에 있는 것 → `cd /d D:\작업폴더\hbi`. 주소가 다르면 메모장으로 돌아가 고침

단계 4. 자료 점검

이 단계에서 할 일

폴더 주소가 맞는지, 필요한 지도 파일이 다 있는지 확인합니다.

1. 아래 상자를 치고 Enter. 글자가 짝 나오고, 맨 아래에 `D:\작업폴더\hbi>` 가 다시 보이면 끝입니다.
2. 그림 ① `shp` 파일 총 `N`개 의 `N`이 0이 아닌지 봅니다.
3. 그림 ② 에서 `sidewalk_cl`, `road_cl`, `building` 앞이 **[있음]** 인지 봅니다. 아래쪽의 `DEM 5m`: 파일 `N`개 도 0이 아닌지 봅니다.
4. 그림 ③ `JIMOK(지목)` 예시 에 `대`, `도` 같은 한글이 보이는지 봅니다 (필지를 받았을 때만 나옴). 그 밖의 글자는 몰라도 됩니다.

하는 일 받은 자료가 제대로 있는지 점검만 합니다 (자료는 바꾸지 않음)

입력 `python 01_inspect.py`

OSGeo4W Shell

```
D:\작업폴더\hbi>python 01_inspect.py

=== 1. 수치지형도 폴더 내 레이어 코드 ===
shp 파일 총 12개 ① (루트: E:\제공자료\수치지형도)

[있음] sidewalk_cl ②: 파일 2개 / 첫 파일 객체 5개 / 좌표계 prj 없음 → 추정 5186
[있음] road_cl ②: 파일 2개 / 첫 파일 객체 27개 / 좌표계 prj 있음 5186

[있음] building ②: 파일 2개 / 첫 파일 객체 250개 / 좌표계 prj 있음 5186

DEM 5m: 파일 2개

JIMOK(지목) 예시: {'대': 1372, '도': 228} ③ ← 한글이 깨지면 config.PARCEL_ENCODING 변경
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

[없음] 이 보이면 아래 표로 판단합니다. 표의 이름(`stairs` 등)은 화면의 **[있음]** · **[없음]** 줄에서 찾습니다 (그림에는 일부만 보임).

화면 문구	뜻 → 할 일
[없음] building , 또는 sidewalk_cl · road_cl 둘 다 [없음]	분석 불가 : 집 또는 길 지도가 없음 → 멈추고 담당자에게 파일 이름 확인
sidewalk_cl · road_cl 중 하나만 [없음] , 또는 [없음] stairs · bus_stop · station	결과 약화 : 길 일부·계단·정류장·역이 빠짐 → 메모하고 계속
[없음] bridge · tunnel · overpass	관찰음 : 다리·터널·육교 → 그대로 계속

✓ 성공하면

① 0이 아님, ② 세 줄 모두 **[있음]** (하나라도 **[없음]** 이면 위 표대로), ③ 한글이 보임 (필지가 "없음"이면 ③은 안 나와도 됨)

✗ 이상하면

① `shp` 파일 총 0개 → 단계 3 `DATA_ROOT_MAP` 주소 확인. `DEM 5m`: 파일 0개 → 단계 3 `DATA_ROOT_DEM` 주소 확인. ③ ??? 처럼 깨지면 **notepad config.py** → Ctrl+F 로 `PARCEL_ENCODING` 을 찾아 `"UTF-8"` 을 `"CP949"` 로 바꾸고 저장, 이 단계를 다시

단계 5. 구역 A 실행

이 단계에서 할 일

분석 범위(계산할 지역)를 구역 A 로 정하고, 명령 한 줄로 분석을 돌립니다.

1. **notepad config.py** 로 메모장을 엽니다. Ctrl+F 로 **AREA_BB0X = None** 줄을 찾습니다.
2. 그 줄을 **통째로** 그림 ① 의 줄로 바꿉니다. 숫자 네 개·쉼표·대괄호 **[]** 를 단계 6 표의 A 줄과 한 글자씩 대조합니다 (틀리면 계산 범위가 달라져 05 에서 멈출 수 있음).
3. Ctrl+S 로 저장하고, 아래 첫 입력 상자 **findstr /b "AREA_BB0X" config.py** 로 확인합니다: 화면에 **정확히 한 줄**, 그림 ② 처럼 A 줄과 같아야 합니다. 두 줄이 나오거나 값이 다르면 메모장으로 돌아가 고칩니다.
4. 아래 둘째 입력 상자의 **python run_all.py** 를 치고 Enter. **끝날 때까지 창을 닫지 않고, 창 안을 클릭하지 않습니다** (제목 앞에 **선택** 이 보이면 Esc). 끝나면 단계 5-2 로.

바꾸기 전

```
config.py - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
AREA_BB0X = None
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

바꾼 후

```
config.py - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
AREA_BB0X = [194500, 548300, 203400, 560200]
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

하는 일 config.py 에서 **AREA_BB0X** 로 시작하는 줄만 보여 줍니다 (확인용). # 예) ... 설명 줄은 나오지 않습니다

입력 **findstr /b "AREA_BB0X" config.py**

확인 화면

```
OSGeo4W Shell
D:\작업폴더\hbi>findstr /b "AREA_BB0X" config.py
AREA_BB0X = [194500, 548300, 203400, 560200]
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

하는 일 이 구역의 분석을 01 ~ 06 단계까지 한 번에 돌립니다

입력 **python run_all.py**

실행 중

```
OSGeo4W Shell
D:\작업폴더\hbi>python run_all.py

##### 01_inspect.py #####
=== 0. 실행 환경 ===
...
##### 02_network.py #####
[16:06:23] 보행 네트워크 구축
...
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ **성공하면**

로 시작하는 줄이 01, 02, 03, 04, 05, 06 순서로 차례차례 나타남

✗ **이상하면**

!! ... 에서 중단 이 나오면 → 단계 8. 구역마다 몇 분에서 수십 분 걸릴 수 있고 (03 이 가장 오래 걸림), 새 글자가 한동안 안 나와도 정상. 1시간이 넘도록 아무 변화가 없으면 메모하고 담당자에게 알림

단계 5-2. 구역 A 끝난 뒤

이 단계에서 할 일

숫자 두 개를 적고, 결과 폴더 이름을 **output_A** 로 바꿉니다.

1. 맨 아래에 **D:\작업폴더\hbi>** 가 다시 보이면 끝난 것입니다 (그림 ④). 그 위에 그림 ③ 완료 → **output/** (반출 신청 대상) 이 있는지 봅니다.
2. 마우스 휠로 창을 위로 올려 두 숫자를 찾아 단계 6 메모 칸 **A** 줄에 적습니다. 화면 어디에도 **!! ...** 에서 중단 이 없으면 ○ 를 적습니다: 그림 ① 최대 연결망 노드 비율 뒤의 % (예: 100.0%), 그림 ② 줄 맨 끝 괄호 안의 % (예: 69.8%).
3. 아래 입력 상자의 **ren output output_A** 를 치고 Enter. 결과 폴더 이름이 바뀝니다 (그림 ⑤).

위로 올려서 보는 부분

```
OSGeo4W Shell
[16:06:23]    끊긴 끝점 연결 2개 → 최대 연결망 노드 비율 100.0% ① (90% 이상이면 정상)
...
[16:06:25]    데이터 경계 500m 이내 건물 278개 → 통계에서 제외 예정 (69.8%) ②
...
[16:06:28]    완료 → output/ (반출 신청 대상) ③

##### 06_parcel.py #####
[16:06:28]    필지 1,600개
...

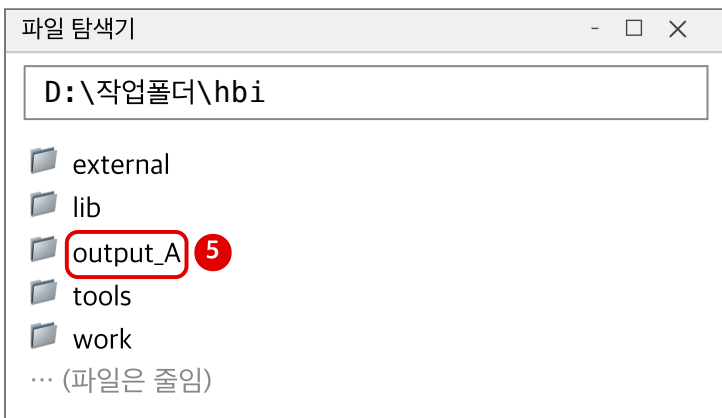
D:\작업폴더\hbi> ④
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

하는 일 결과 폴더 **output** 의 이름을 **output_A** 로 바꿉니다 (다음 구역 결과와 섞이지 않게)

입력 **ren output output_A**

명령 대신 파일 탐색기로 해도 됩니다: **hbi** 폴더에서 **output** 폴더를 마우스 오른쪽 클릭 → 이름 바꾸기 → **output_A** 입력 → Enter.



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

③ 완료 → **output/** (반출 신청 대상) 이 있음. ⑤ **hbi** 폴더에 **output_A** 가 생김

✗ 이상하면

!! ... 에서 중단 이 보이면 × 로 적고 단계 8 표. ① 이 90% 보다 낮으면 메모만 하고 계속. ② 가 **100%** 면 계산할 집이 없어 05 에서 멈춤 → 단계 8 표 넷째 줄

단계 6. 구역 B · C · D

이 단계에서 할 일

단계 5-5-2 와 똑같이 합니다. 바꾸는 것은 메모장의 AREA_BB0X 줄과 결과 폴더 이름뿐입니다.

1. **notepad config.py** → Ctrl+F 로 AREA_BB0X 를 찾으면 # 으로 시작하는 예시 줄(# 예) 관악구 일부 ...)이 먼저 나옵니다 → Enter 를 한 번 더 눌러 다음 줄로. # 이 없는 AREA_BB0X = ... 줄을 아래 표의 그 구역 줄로 **통째로** 바꾸고 Ctrl+S. 시작하는 구역의 □ 에 체크합니다 (A□ 는 단계 5에서 한 구역이므로 이미 끝났으면 체크).
2. **findstr /b "AREA_BB0X" config.py** 로 확인: **정확히 한 줄**, 그 구역 표의 줄과 같음. 아래 그림처럼 두 줄이 나오거나 앞 구역 값이 나오면 메모장으로 돌아가 고칩니다 (그대로 돌리면 오류 없이 앞 구역을 다시 계산합니다).
3. **python run_all.py** 를 치고 Enter, 끝날 때까지 기다림 (창 안 클릭하지 않기).
4. 두 숫자와 끝났는지를 아래 메모 칸에 적음: 화면에 !! ... 에서 중단 이 없으면 ○, 있으면 × 와 그 오류의 마지막 줄.
5. **ren output output_B** (C 구역이면 output_C, D 구역이면 output_D).

구역	동네	메모장에 넣을 줄
A□	종로구·중구	AREA_BB0X = [194500, 548300, 203400, 560200]
B□	관악구	AREA_BB0X = [190100, 536300, 200000, 545000]
C□	광진구	AREA_BB0X = [203900, 546100, 211100, 553700]
D□	강서구	AREA_BB0X = [178200, 546400, 190500, 557200]

잘못 고친 예 (구역 B 인데 # 예시 줄을 바꿔 두 줄이 나옴 → 다시 고치기)

```
OSGeo4W Shell
D:\작업폴더\hbi>findstr /b "AREA_BB0X" config.py
AREA_BB0X = [190100, 536300, 200000, 545000]
AREA_BB0X = [194500, 548300, 203400, 560200] 1
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

구역	① 최대 연결망 노드 비율	② 데이터 경계 500m 이내 (x%)	끝났나 ○/×	×일 때 오류 마지막 줄
A				
B				
C				
D				

✓ 성공하면

네 구역 모두 ○, hbi 폴더에 output_A ~ output_D 네 개

✗ 이상하면

한 구역이 ×면 단계 8 표를 봅니다. 표대로 해도 안 풀리면 그 구역의 오류 마지막 줄을 적고, **ren output output_B 실패** 처럼 이름을 바꾼 뒤 다음 구역으로 (hbi 폴더에 output 이 없으면 이름 바꾸기는 건너뛸)

단계 7. 마무리

이 단계에서 할 일

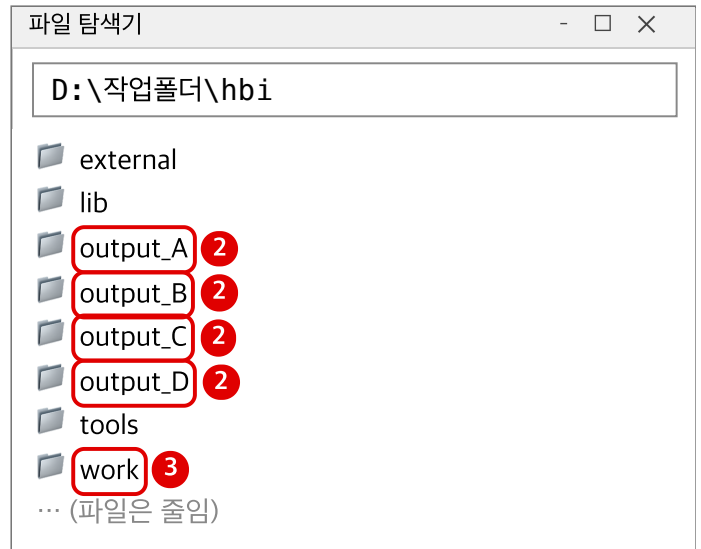
설정을 원래대로 돌리고, 가지고 나갈(반출할) 폴더를 확인합니다.

1. **notepad config.py** → Ctrl+F 로 # 이 없는 AREA_BB0X 줄을 찾아 AREA_BB0X = None 으로 되돌리고 Ctrl+S (그림 ①). **findstr /b "AREA_BB0X" config.py** 로 확인하면 AREA_BB0X = None 한 줄만 나와야 합니다.
2. 파일 탐색기에서 hbi 폴더를 엽니다. 담당자에게 그림 ② 네 폴더를 반출 신청하겠다고 말합니다 (신청 방법은 담당자가 알려 줌).
3. 그림 ③ work 폴더는 반출하지 않습니다 (건물 하나하나의 결과가 들어 있어 반출 금지). output_A 실패 처럼 실패한 폴더도 반출하지 않습니다.
4. external, lib, tools 폴더와 다른 파일은 그대로 둡니다 (반출하지 않음). 아래 "메모할 것"을 확인합니다.

되돌린 후



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

메모할 것:

- 단계 6 메모 칸의 숫자 (구역마다 두 개)
- 멈춘 명령이 있으면 그 명령과 화면 **마지막 줄** (그대로 옮겨 적기)
- 단계 4 점검에서 [없음] 이나 깨진 글자처럼 예상과 다른 줄

✓ 성공하면

AREA_BB0X = None 으로 저장됨, 반출 신청한 폴더가 output_A ~ output_D

✗ 이상하면

반출할 폴더가 네 개보다 적으면, 빠진 구역과 그 이유를 메모

단계 8. 막혔을 때

이 단계에서 할 일

화면에 나온 글자로 아래 표에서 찾아 그대로 합니다. D:\작업폴더 자리에는 단계 0 ④ 주소를 칩니다.

화면에 나온 것	뜻	할 일
'python'은(는) 내부 또는 외부 명령... 이 아닙니다	일반 명령창에 쳤음	단계 1처럼 OSGeo4W Shell 을 다시 열고, 하던 단계의 첫 명령부터 다시 (단계 2 이면 <code>cd /d D:\작업폴더</code> , 그 뒤 단계면 <code>cd /d D:\작업폴더\hbi</code>)
can't open file ... No such file or directory	hbi 폴더 밖에 있음	<code>cd /d D:\작업폴더\hbi</code> 로 이동한 뒤, 멈춘 명령을 다시
06 에서 지목 '대' 필지가 0개입니다	필지 한글이 깨짐	이름 바꾸기 전에: <code>notepad config.py</code> → <code>PARCEL_ENCODING = "CP949"</code> 로 바꾸고 저장 → <code>python 06_parcel.py</code> 한 줄만 다시 (02~05 는 필지와 상관없음) → 단계 5-2 처럼 이름 바꾸기. 이미 이름을 바꿨으면 먼저 <code>ren output_A output</code> 으로 되돌림. 이미 "CP949" 였으면 그 줄을 메모하고 그대로 이름 바꾸기
창 제목 앞에 선택	창 안을 클릭해 선택 상태로 멈춤	Esc 를 누르면 이어서 실행됨
SyntaxError 와 utf-8 이 보임 (예: SyntaxError: (unicode error) 'utf-8' codec can't decode byte ...)	메모장이 config.py 를 UTF-8 이 아닌 형식으로 저장함	<code>notepad config.py</code> → 파일 → 다른 이름으로 저장 → 아래 인코딩을 UTF-8 → 같은 이름 <code>config.py</code> 로 저장(바꾸기 예) → 멈춘 명령을 다시
05 에서 min() iterable argument is empty	분석 범위 안에 계산할 집이 0채	메모장의 AREA_BB0X 줄이 단계 6 표와 한 글자씩 같은지 확인. 틀렸으면 고쳐 저장하고 <code>python run_all.py</code> 다시. 같은데도 나오면 <code>ren output output_A 실패</code> (B 구역이면 <code>output_B_실패</code>) 하고 다음 구역

config.py 안의 **EDGE_BUFFER** 줄은 바꾸지 않습니다 (자료 가장자리 500m 를 빼는 설정입니다). 그림의 빨간 ① 이 표의 "화면에 나온 것"이고, ② 는 어느 단계에서 멈췄는지 알려 주는 줄입니다.

오류 예시 1: 일반 명령창(명령 프롬프트)에 쳤을 때

```

명령 프롬프트
D:\작업폴더\hbi>python run all.py
'python'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는
배치 파일이 아닙니다. ①

```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

오류 예시 2: 계산할 집이 0채일 때

```

OSGeo4W Shell
##### 05_export.py #####
Traceback (most recent call last):
  File "D:\작업폴더\hbi\05_export.py", line 94, in <module>
    ax.set_xlim(min(xs) * C.GRID, (max(xs) + 1) * C.GRID)
ValueError: min() iterable argument is empty ①
!! 05_export.py 에서 중단. 위 오류 메시지를 확인하세요. ②

```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

표에 없는 오류가 나오면 여기서 멈추고, 화면의 오류 마지막 줄을 그대로 적어 옵니다. (표에 있는 오류는 위 "할 일" 대로 하고, 그래도 안 풀린 구역만 `_실패` 로 두고 다음 구역으로 갑니다.)

3부. 자세한 설명

왜 구역을 넷으로 나눠 돌리나

v4 분석 코드는 길 연결망 가운데 가장 큰 덩어리 하나만 남깁니다(lib/qnetwork.py ⑧). 5개 구는 서로 떨어진 네 묶음(종로·중 / 관악 / 광진 / 강서)이라, 한 번에 돌리면 한 묶음만 남고 나머지 구의 집은 통째로 빠집니다. 공개 도보 네트워크로 재 보면 가장 큰 덩어리만 남길 때 노드의 38.2%만 남습니다. v4 코드는 반입 심의를 받은 그대로라 고칠 수 없으니, config.py 의 AREA_BBOX 만 바꿔 구역마다 따로 돌립니다 (v5 에서는 코드로 고침).

구역별 분석 범위 (EPSG:5186, 대상 구 범위 + 1,000m, 100m 단위로 바깥쪽 반올림)

구역	대상 구	config.py 에 넣을 줄	크기	들어 있는 선정지 (잘린 선까지 거리)
A	종로구·중구	AREA_BBOX = [194500, 548300, 203400, 560200]	8.9 × 11.9 km	송인동(2,068m), 신당동(1,949m), 대현산배수지공원 구간
B	관악구	AREA_BBOX = [190100, 536300, 200000, 545000]	9.9 × 8.7 km	봉천동(2,631m)
C	광진구	AREA_BBOX = [203900, 546100, 211100, 553700]	7.2 × 7.6 km	중곡동 무지개계단(1,881m)
D	강서구	AREA_BBOX = [178200, 546400, 190500, 557200]	12.3 × 10.8 km	화곡동(2,786m)

- 1,000m 를 넓힌 이유: 구 경계 너머의 병원·정류장까지 경로에 들어가게 하려고.
- v4 는 bbox 밖을 "데이터 없음"으로 보고, 잘린 선에서 500m(EDGE_BUFFER) 안의 건물을 통계에서 자동으로 뺍니다(lib/qedge.py). 그래서 대상 구 안 건물은 잘린 선에서 1,000m 이상 떨어져 있어 빠지지 않고, 선정지 반경 300m 원도 잘린 선에서 모두 1,800m 이상입니다.
- 넓힌 띠(대상 구 밖) 건물 일부는 결과에 들어가지만, 밖에서 tools/merge_exports.py 가 대상 구 안 격자·동만 남겨 합칩니다. 구역 bbox 끼리는 겹치지 않습니다 (A 는 x 203,400 까지, C 는 203,900 부터).

run_all.py 가 하는 일

- python run_all.py** 는 01 → 02 → 03 → 04 → 05 를 차례로 돌리고, DATA_ROOT_PARCEL 에 필지 경로가 있으면 06 을, SKT·KCB 경로(JOIN_DATA)가 있으면 07 을 이어서 돌립니다. 민감도 분석(08)은 --sens 를 붙일 때만 들어가므로, 시간이 되면 이름 바꾸기 전에 **python 08_sensitivity.py** 를 따로 칩니다.
- 걸리는 시간**: 02(길 네트워크 만들기)는 구역 A 크기의 공개 도보 네트워크(링크 58,560개 → 노드 97,124개)로 v4 코드를 이 맥에서 돌려 보니 4~16초였습니다 (QGIS 4.2.3·3.32 조건, scipy 있음·없음 모두). 03(경로 계산)은 건물 자료가 없어 재지 못했습니다. 건물 수·목적지 수에 따라 가장 오래 걸리는 단계이고, 안심구역 컴퓨터가 이 맥보다 느릴 수 있으므로 구역마다 수십 분까지 잡습니다.
- AREA_BBOX 확인이 필요한 이유**: config.py 57행에 # 예) 관악구 일부 2km×2km: AREA_BBOX = [194000, 541000, 196000, 543000] 설명 줄이 있고, 실제 설정은 59행 AREA_BBOX = None 입니다. 57행을 고치면 59행이 앞 구역 값 그대로라 오류 없이 앞 구역을 다시 계산합니다. findstr /b 는 줄 맨 앞이 AREA_BBOX 인 줄만 보여 주므로 # 설명 줄은 나오지 않습니다. 리허설에서 57행을 통째로 바꾸면 두 줄, 57행의 숫자만 바꾸면 앞 구역 값 한 줄이 나와 둘 다 확인에서 걸렸습니다.
- 한 단계에서 오류가 나면 거기서 멈추고 !! <단계>.py 에서 중단. 위 오류 메시지를 확인하세요. 가 나옵니다 (리허설에서 확인).
- 5줄(02·03·06·04·05)로 따로 돌린 것과 결과가 같습니다: 리허설에서 output CSV·PNG 11개 md5 동일, gpkg 3개는 저장 시각만 다름. 06 은 04·05 결과를 바꾸지 않아서 마지막에 돌아도 됩니다.
- 중간에 멈췄을 때**: 원인을 고친 뒤 멈춘 단계부터 한 줄씩 다시 칠 수 있습니다 (앞 단계 결과는 work 폴더에 남아 있음). 순서: **python 02_network.py** → **python 03_hbi.py** → **python 04_validate.py** →

`python 05_export.py` → (필지가 있으면) `python 06_parcel.py`. 단 AREA_BB0X 나 데이터 경로를 고쳤으면 02 부터(또는 `python run_all.py` 로 처음부터) 다시.

화면에서 확인할 기준 (구역마다)

단계	화면 문구	기준
02	최대 연결망 노드 비율 N%	90% 이상 이 정상 (한 구역만 돌리므로). 한참 낮으면 그 구역 안에서 길이 갈라진 것 → 메모만 하고 계속
03	건물 N개 중 주거 M개 (네트워크 연결 N%)	95% 이상
03	데이터 경계 500m 이내 건물 N개 → 통계에서 제외 예정 (x%)	넓힌 때 때문에 평소보다 높게 나와도 정상. EDGE_BUFFER 를 줄이지 않음 (README 의 "50% 넘으면 300으로" 는 구역별 실행에는 적용 안 함). 100% 면 계산할 집이 없어 05 가 멈춤
05	완료 → output/ (반출 신청 대상)	이 줄이 있어야 성공
06	필지 N개, 그중 지목 '대' 필지 수	0 이 아님. 지목 '대' 필지가 0개입니다 면 PARCEL_ENCODING 을 "CP949" 로 바꾸고 이름 바꾸기 전에 <code>python 06_parcel.py</code> 만 다시

- 04 의 선정지·구간은 5곳·전 구간을 구역마다 계산해도 됩니다 (합칠 때 그 구역 것만 씀). 06 은 그 구역 건물 범위 안의 필지만 읽습니다.
- 05 가 `min() iterable argument is empty` 로 멈추면 그 구역의 계산할 집이 0채입니다 (v4·v5 공통, 코드는 고치지 않음 → docs/known_issues.md). 멈추기 전까지 output 에는 04 결과 일부만 남으므로 output_A_실패 처럼 이름을 바꿔 두고 반출하지 않습니다.

반출 규칙

- output_A, output_B, output_C, output_D 네 폴더만 반출 신청합니다 (모두 격자·행정동 집계, 건물 5개 미만 제외 — v4 반출 규칙 그대로).
- work 는 건물 하나하나의 결과라 반출하지 않습니다. 실패한 구역 폴더(output_?_실패)도 반출하지 않습니다.
- 네 구역을 다 돌린 뒤 AREA_BB0X = None 으로 되돌려 둡니다.

결과를 받은 뒤 (안심구역 밖, 이 맥에서)

1. 반출받은 네 폴더를 results/raw_export_zones/A ~ D 에 그대로 넣습니다 (Finder 로 복사. output_A 의 파일들 → A, output_B → B ...).
2. 구역 결과를 하나로 합칩니다.

하는 일 구역 결과 네 개를 하나로 합쳐 results/raw_export 에 만듭니다

입력

```
python3 tools/merge_exports.py \
  --zone A=results/raw_export_zones/A --zone B=results/raw_export_zones/B \
  --zone C=results/raw_export_zones/C --zone D=results/raw_export_zones/D
```

3. 고령인구를 붙입니다.

하는 일 행정동별 65세 이상 인구를 붙여 고령인구 추정표를 만듭니다

입력

```
python3 tools/outside_elderly.py results/raw_export/dong_hbi.csv
```

4. 기획서를 만듭니다.

하는 일 실제 결과로 기획서(발표 자료)를 만듭니다

입력

```
cd deck && npm run build:real
```

합칠 때 무엇을 어떻게 근사했는지는 results/raw_export/merge_notes.txt 에 적힙니다.

- summary 는 대상 구 격자로 다시 씩니다. 건물 5개 미만 격자는 원래 반출에 없으므로 그만큼 적게 씩니다. 중 양값은 합칠 수 없어 빠집니다.
- 선정지 백분위와 신규 후보는 합친 격자 전체 기준으로 다시 계산합니다.

- 기여도·민감도는 구역 값을 분석 건물 수로 가중 평균한 근사입니다.
- 구간(validation_routes)은 출발·도착이 그 구역 bbox 안, 잘린 선에서 500m 이상 안쪽인 구역 값만 씁니다.

4부. 부록: 맥 리허설

목요일 안심구역(Windows, QGIS 3.32)에서 할 일을 이 맥에서 가짜 데이터로 해 봅니다. 쓰는 번들은 v4 (deliverables/hbi_code_bundle_v4.txt, 1차 방문에 쓰는 그 파일)입니다. 아래 상자의 명령을 터미널에 한 줄씩 복사해 붙여 넣고 Enter.

1. 터미널과 QGIS Python 준비

Command + Space → 터미널 → Enter. 이 맥의 QGIS 4.2.3 Python 을 python 이라는 이름으로 부르게 하는 두 줄 (터미널 창을 새로 열면 다시 붙여 넣기):

하는 일 QGIS 가 설치된 위치를 Q 라는 이름으로 기억시킵니다

입력 `Q=/Applications/QGIS-final-4_2_3.app/Contents`

하는 일 python 이라고 치면 QGIS 안의 Python 이 실행되도록 설정합니다 (터미널 창을 새로 열면 다시)

입력

```
python() { PYTHONHOME=$Q/Resources
PYTHONPATH=$Q/Resources/python3.12:$Q/Resources/python3.12/lib-
dynload:$Q/Resources/python3.12/site-packages GDAL_DATA=$Q/Resources/qgis/gdal
PROJ_DATA=$Q/Resources/qgis/proj $Q/MacOS/python3.12 "$@"; }
```

확인 (3.12.11 3.13.3 처럼 나오면 준비 끝):

하는 일 Python 과 GDAL 이 제대로 불러와지는지 버전만 찍어 봅니다

입력 `python -c "import sys; from osgeo import gdal; print(sys.version.split()[0], gdal.__version__)"`

2. 번들 풀기와 가짜 데이터

하는 일 연습 폴더를 만들고, 번들을 복사한 뒤, 그 폴더로 들어갑니다

입력

```
mkdir -p ~/evenly_rehearsal && cp
~/Downloads/evenly/deliverables/hbi_code_bundle_v4.txt ~/evenly_rehearsal/ && cd
~/evenly_rehearsal
```

하는 일 우리 파일(번들)을 풀어 hbi 폴더를 만듭니다

입력 `python hbi_code_bundle_v4.txt`

하는 일 hbi 로 들어가 연습용 가짜 자료(지도·높이·필지)를 만듭니다

입력 `cd hbi && python tools/make_test_data.py`

→ 완료: 파일 43개 와 완료: .../hbi/testdata (가짜 지형도·DEM·필지. 안심구역에서는 이 단계가 없습니다)

3. config.py 고치기

`code config.py` 로 엽니다 (VS Code 가 없으면 `open -a TextEdit config.py`, 메뉴 편집 → 대치 → "스마트 따옴표" 끄기). 아래 줄을 줄 전체 바꾸고 저장:

찾을 줄 (처음 모습)	바꿀 내용
<code>DATA_ROOT_MAP = r"D:\data\수치지형도"</code>	<code>DATA_ROOT_MAP = r"testdata/map"</code>
<code>DATA_ROOT_DEM = r"D:\data\DEM5m"</code>	<code>DATA_ROOT_DEM = r"testdata/dem"</code>
<code>DATA_ROOT_PARCEL = None</code>	<code>DATA_ROOT_PARCEL = r"testdata/parcel"</code>
<code>AREA_BBOX = None</code>	<code>AREA_BBOX = [200000, 545000, 201000, 546000] (연습 구역 1)</code>
<code>EDGE_BUFFER = 500</code>	<code>EDGE_BUFFER = 100 (연습에서만! 안심구역에서는 500 그대로)</code>

연습 구역 2 는 AREA_BB0X = [200000, 546000, 201000, 547000] 입니다 (가짜 데이터 x 200,000~201,000, y 545,000~547,000 을 위아래 반으로 나눔). EDGE_BUFFER 를 연습에서만 100 으로 두는 이유: 가짜 데이터를 반으로 나누면 1×1 km 라, 500m 를 빼면 남는 집이 0채가 되어 05 가 min() iterable argument is empty 로 멈춥니다 (실제로 해 봄). 진짜 구역은 1,000m 를 넓혀 잘라 이런 일이 없습니다. 확인 (다섯 줄이 표의 "바꿀 내용" 대로):

하는 일 config.py 에서 고친 다섯 줄만 골라 보여 줍니다 (확인용, 아무것도 바꾸지 않음)

입력

```
grep -n
"^DATA_ROOT_MAP\|^DATA_ROOT_DEM\|^DATA_ROOT_PARCEL\|^AREA_BB0X\|^EDGE_BUFFER"
config.py
```

4. 구역 1 → 구역 2 (안심구역과 같은 손동작)

하는 일 연습 구역 1 의 분석을 01 ~ 06 단계까지 한 번에 돌립니다

입력

```
python run_all.py
```

하는 일 결과 폴더 이름을 output_A 로 바꿉니다 (맥은 ren 대신 mv)

입력

```
mv output output_A
```

그다음 code config.py 로 AREA_BB0X 줄만 연습 구역 2 로 바꾸고 저장한 뒤:

하는 일 연습 구역 2 의 분석을 한 번에 돌립니다

입력

```
python run_all.py
```

하는 일 결과 폴더 이름을 output_B 로 바꿉니다

입력

```
mv output output_B
```

볼 줄	연습 구역 1	연습 구역 2
02 최대 연결망 노드 비율	100.0%	100.0%
03 건물 N개 중 주거 M개 (네트워크 연결 N%)	116개 중 주거 91개, 100.0%	135개 중 주거 111개, 100.0%
03 데이터 경계 100m 이내 건물	29개 (31.9%)	36개 (32.4%)
05 완료 → output/	나옴 (행정동 2개)	나옴 (행정동 3개)
06 필지 N개 / '대'	462 / 50	441 / 54
output 파일 수	14개	12개 (가짜 정류장이 남쪽에만 있어 bus 지도·gpkg 없음. 정상)

다시 연습하려면 cd ~ 한 뒤 rm -rf ~/evenly_rehearsal 로 지우고 2번부터.

5. 안심구역(Windows)에서 다른 점

항목	이 맥 (리허설)	안심구역 (Windows, QGIS 3.32)
명령창	터미널 + 1번의 <code>python()</code> 두 줄	시작 메뉴 → OSGeo4W Shell. 설정 없이 바로 <code>python</code>
폴더 이동	<code>cd ~/evenly_rehearsal/hbi</code>	<code>cd /d D:\작업폴더\hbi</code> (드라이브가 다르면 <code>/d</code> 가 있어야 이동됨)
가짜 데이터	<code>python tools/make_test_data.py</code>	하지 않음 (진짜 데이터)
편집기	<code>code config.py</code> 또는 TextEdit	<code>notepad config.py</code> (메모장). 저장할 때 인코딩 UTF-8 유지
데이터 경로	<code>r"testdata/map"</code> (슬래시)	<code>r"E:\...\수치지형도"</code> 처럼 역슬래시 + 앞에 <code>r</code> . <code>r</code> 을 빼면 <code>\d, \n</code> 이 특수문자로 읽혀 경로가 깨짐
AREA_BBOX	연습 구역 1·2	1부 표의 A·B·C·D 네 줄
EDGE_BUFFER	연습에서만 100	500 그대로
폴더 이름 바꾸기	<code>mv output output_A</code>	<code>ren output output_A</code>
설정 확인	<code>grep -n "^AREA_BBOX" config.py</code>	<code>findstr /b "AREA_BBOX" config.py</code>